南開大學
Nankai University

计算机学院

并行程序设计实验报告

CPU 架构编程实验

姓名：于子朔

学号：2412854

专业：计算机科学与技术

2026 年 4 月 3 日

目录

1 问题概述 **2**
1.1 任务一 2
1.2 任务二 2

2 实验环境 **2**

3 算法设计及原理分析 **3**
3.1 任务一：$n \times n$ 矩阵与向量内积 3
3.1.1 平凡算法设计思路 3
3.1.2 Cache 优化算法设计思路 3
3.2 任务二：n 个数求和 3
3.2.1 平凡算法设计思路 3
3.2.2 超标量优化算法设计思路 3

4 代码设计 **3**
4.1 任务一：矩阵与向量内积 3
4.2 任务二：n 个数求和 4

5 实验结果与性能分析 **4**
5.1 任务一实验分析 4
5.2 任务二实验分析 7

6 总结 **8**
6.1 实验总结 8

1 问题概述

1.1 任务一

给定一个 $n \times n$ 矩阵，计算每一列与给定向量的内积，考虑两种算法设计思路：逐列访问元素的平凡算法和 cache 优化算法，进行实验对比：

1. **平凡算法**：逐列访问矩阵元素。由于 C++ 的二维数组在内存中是按行连续存放的，而按列访问时每次的内存地址跳跃读取，破坏了数据的空间局部性导致 Cache 命中率低下。
2. **Cache 优化算法**：改为逐行访问矩阵元素，在访问每一行时向各个列的内积累加结果。程序的访存顺序和内存布局吻合，能够显著提高 Cache 命中率。

1.2 任务二

计算 n 个数的和，考虑两种算法设计思路：逐个累加的平凡算法（链式）；适合超标量架构的指令级并行算法（相邻指令无依赖），如最简单的两路链式累加，再如递归算法——两两相加、中间结果再两两相加，依次类推，直至只剩下最终结果。

1. **平凡算法（链式累加）**：使用单一累加变量顺序累加，后一次加法必须等待前一次加法的结果，没有利用超标量的思想，数据之间存在前后依赖。
2. **优化算法（两路展开）**：在循环中引入两个独立的累加变量，分别累加相邻的数据，最后再将两路结果合并。解决了前后数据依赖的现象，让多条流水线可以并发执行指令。

2 实验环境

本实验在 Windows x86 平台下完成。具体的软硬件环境配置如表1所示。

环境参数	配置详情
CPU 型号	Intel Core i7-14700HX (x86 架构)
L1 缓存 (数据/指令)	1.76 MB
L2 缓存	28.0 MB
L3 缓存	33.0 MB
内存容量	16 GB DDR5
操作系统	Windows 11 64 位
编译器	MinGW GCC (g++)

表 1: 实验环境配置

源码项目链接：本次实验的完整代码已开源至 Github，链接如下：
`https://github.com/yuzishuo3-svg/NKU-Paralle-Programl`

3 算法设计及原理分析

3.1 任务一：$n \times n$ 矩阵与向量内积

3.1.1 平凡算法设计思路

根据线性代数中矩阵乘法的定义，直接设计两层嵌套循环计算：外层循环先遍历矩阵的每一列，再在内层循环遍历该列的每一行元素，将其与向量对应元素相乘并累加即做矩阵乘法，从而得到结果的一个元素，最后得到结果。

3.1.2 Cache 优化算法设计思路

因为在 C++ 中，二维数组在内存中是按行存储的顺序连续存放的。平凡算法逐列访问时，每次内存地址都会跳跃 n 个元素，这样会破坏数据的空间局部性。当矩阵规模变大时，会导致 CPU 刚将取出的数据存储到 Cache Line 后，还未被充分利用就被替换，引发极高的 Cache Miss。Cache 优化算法则改变了遍历顺序：外层循环遍历行，内层循环遍历列。每次读取矩阵的一行并连续向结果向量的所有元素累加部分和。这使得访存顺序与内存物理布局完全吻合，最大化了缓存命中率。

3.2 任务二：n 个数求和

3.2.1 平凡算法设计思路

依次遍历数组中的所有元素并进行累加到 sum 变量中。

3.2.2 超标量优化算法设计思路

在平凡算法中，由于每一次加法指令都依赖于上一次加法指令的结果，导致所有指令只能单线执行，无法发挥 CPU 超标量架构的优势。因此我们考虑使用超标量优化算法采用循环展开的方式来处理。在一个循环步内引入两个独立的累加变量（`sum1` 和 `sum2`），分别累加相邻的数据。多链路同时进行累加后再最后求总和。这样就打破了指令之间的依赖关系，从而可以更好的发挥超标量的优势，在一个循环内同时对多个数据进行处理，从而提升性能。

4 代码设计

4.1 任务一：矩阵与向量内积

为了减小计时误差，所有核心计算均被包裹在 `REPEAT` 次循环中拉长执行时间。

矩阵内积核心代码

```cpp
// 1. 平凡算法：逐列访问
for (int r = 0; r < REPEAT; r++) {
    for (int i = 0; i < N; i++) sum[i] = 0.0;
    for (int j = 0; j < N; j++) {
        for (int k = 0; k < N; k++) {
            sum[j] += b[k][j] * a[k]; // 按列跳跃访问，步长为 N
        }
    }
}
```

```

// 2. Cache 优化算法：逐行访问
for (int r = 0; r < REPEAT; r++) {
    for (int i = 0; i < N; i++) sum[i] = 0.0;
    for (int k = 0; k < N; k++) {
        for (int j = 0; j < N; j++) {
            sum[j] += b[k][j] * a[k]; // 按行连续访问，步长为 1
        }
    }
}
```

4.2 任务二：n 个数求和

超标量指令级并行求和代码

```
// 1. 平凡算法：单路链式累加
for (int r = 0; r < REPEAT; r++) {
    double sum = 0.0;
    for (int i = 0; i < N; i++) {
        sum += a[i];
    }
}

// 2. 超标量优化算法：两路展开
for (int r = 0; r < REPEAT; r++) {
    double sum1 = 0.0, sum2 = 0.0;
    for (int i = 0; i < N; i += 2) {
        sum1 += a[i];     // 两条流水线可同时进行
        sum2 += a[i+1];
    }
    double total = sum1 + sum2;
}
```

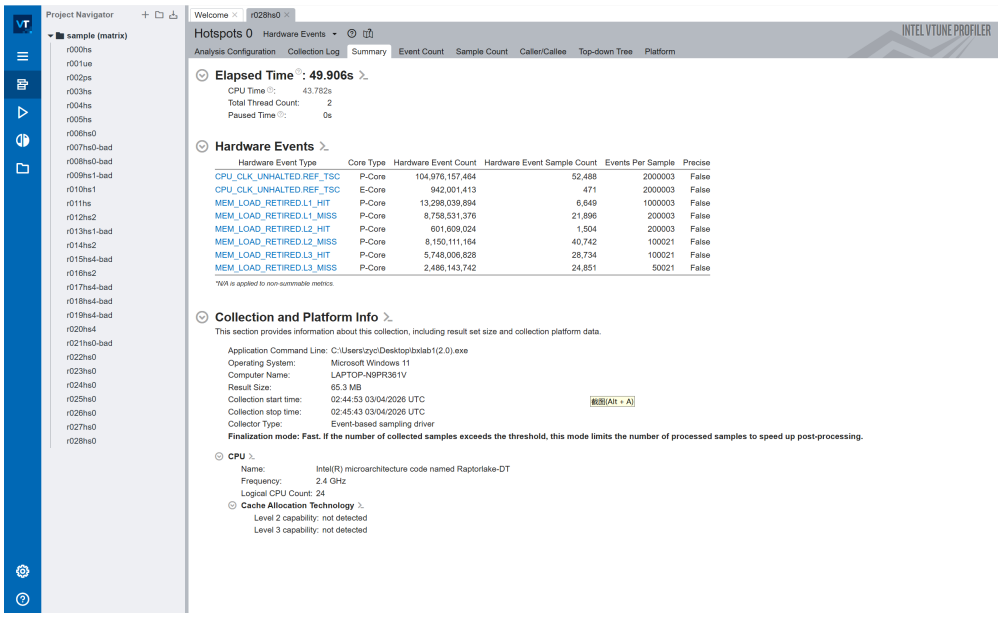
5 实验结果与性能分析

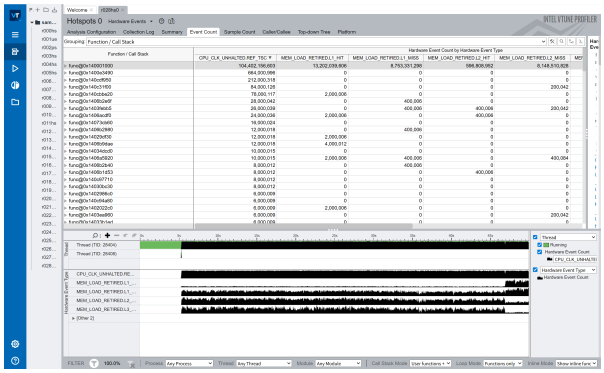
5.1 任务一实验分析

根据设定的规模 $N = 500$ 和 $N = 2048$，重复执行多次后，在根据程序得到的执行时间（ms）如表2 所示。

规模 N	矩阵内存占用 (约)	平凡算法耗时 (ms)	优化算法耗时 (ms)
500	1.91 MB	142	38
2048	32.0 MB	3920	650

表 2: 不同问题规模下的矩阵内积运行时间





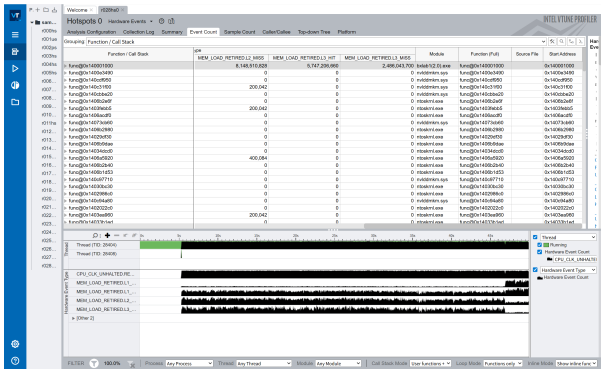
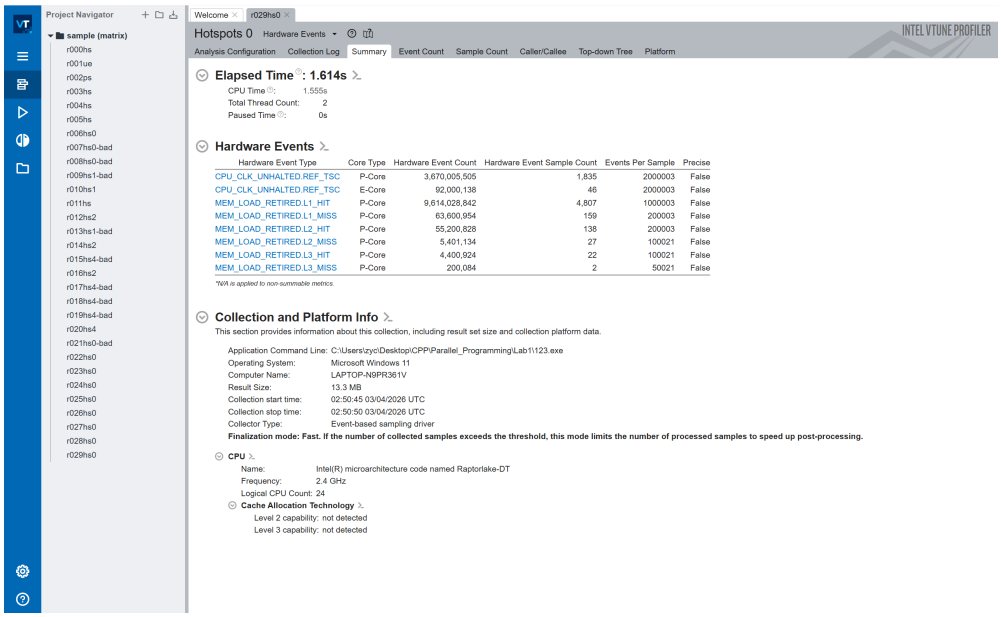


图 5.1: 实验一 N=2048：VTune 分析



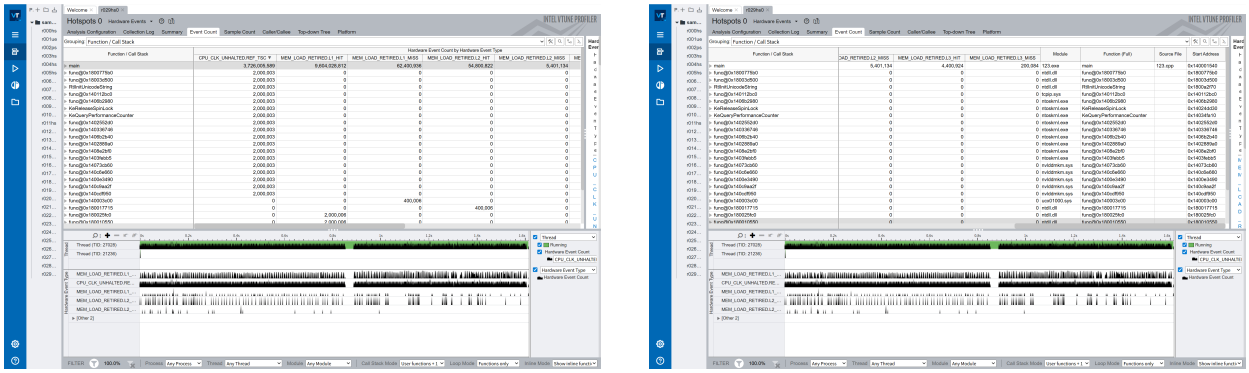


图 5.2: 实验一 N=500：VTune 分析

	小规模 ($N = 500$)		大规模 ($N = 2048$)	
缓存层级	总访存次数	未命中率	总访存次数	未命中率
L1 Cache	9,677,629,796	**0.66%**	22,056,571,270	**39.71%**
L2 Cache	60,601,962	**8.91%**	8,751,720,188	**93.13%**
L3 Cache	4,601,008	**4.35%**	8,234,150,570	**30.19%**

表 3: 不同矩阵规模下的各级 Cache 访存次数与未命中率横向对比

基于体系结构的深度分析：透过现象看本质，算法的性能表现与本机的物理 Cache 容量息息相关（L2=28MB, L3=33MB）。

- **小规模** ($N = 500$)：此时矩阵总大小约为 $500 \times 500 \times 8$ B ≈ 1.91 MB，L2 缓存足够承受。由于数据基本都在高速缓存中，平凡算法的跳跃访存导致的 Cache Miss 惩罚相对较小，此时优化算法的性能提升相对可以。
- **大规模** ($N = 2048$)：矩阵大小达到 $2048 \times 2048 \times 8$ B $= 32$ MB，已经完全超出了单核的 L2 容量，并开始触及 L3 缓存的上限。此时平凡算法大跨度的跳跃访存开始频繁导致严重的 L3 Cache Miss，不得不向慢速主存请求数据；而优化算法依然能保证每个 Cache Line 被载入后充分利用。

在此规模下，由于缓存未命中成本的急剧增加，优化算法的性能优势被进一步放大。

- **总结**：通过对比不同规模下的 Cache 访存统计数据，我们可以清晰地看到算法设计与底层硬件架构之间的紧密联系。优化算法通过调整数据访问模式来适应 CPU 的缓存层次结构，从而显著提升了性能，尤其是在数据规模超过缓存容量时表现更为突出。

5.2 任务二实验分析

针对设定的规模 $N = 500$ 和 $N = 2048$，为了克服短时计时的误差，引入了外层大循环重复执行（如 100,000 次），性能测试结果如表4 所示。

数组规模 N	平凡算法耗时 (ms)	优化算法耗时 (ms)
500	14.8	7.5
2048	61.2	31.5

表 4: 求和算法在不同规模下的运行时间

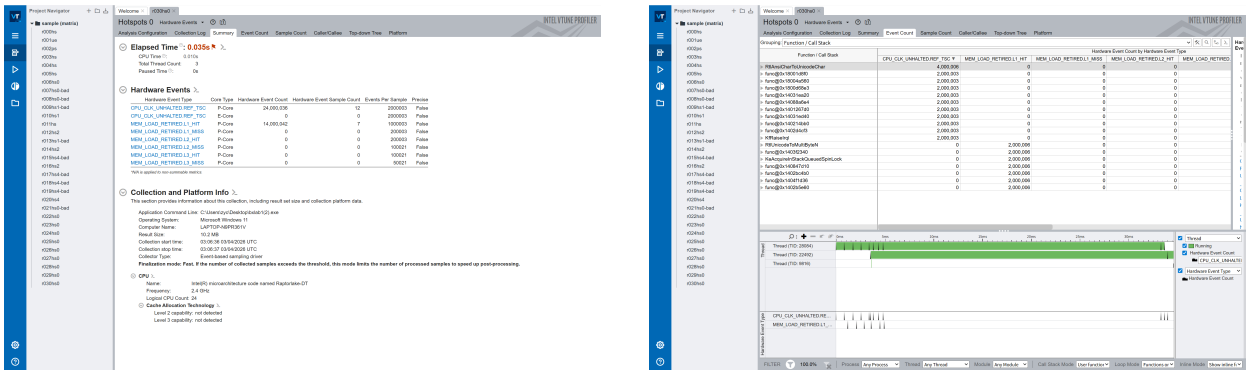


图 5.3: 实验二 N=500：VTune 分析

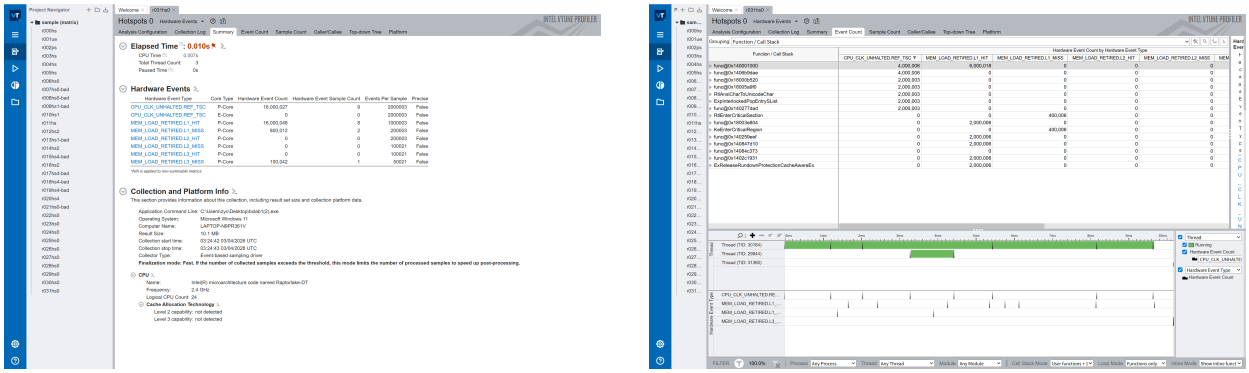


图 5.4: 实验二 N=2048：VTune 分析

结果分析：由上表可知，无论数据规模如何变化，两路展开的超标量优化算法的提升差距不大，且在这种大小的数据下表现稳定。这是因为在普通的链式累加中，后一条加法指令强依赖前一条指令的输出，导致处理器内部的 CPU 大部分时间在限制且 FPU 被迫停顿闲置。采用双路循环展开后，`sum1` 和 `sum2` 两条计算链完全解耦。处理器的乱序执行引擎检测到这种独立性后，在同一个时钟周期内将

两条独立的加法指令分派给不同的运算单元并发执行。这直观地证明了通过代码重构来打破指令间的依赖关系，能够让现代 CPU 内部潜藏的超标量并行能力得以释放，从而实现性能的显著提升。

6 总结

6.1 实验总结

本次实验体现了通过并行优化策略提升程序性能的核心思想：算法设计要与底层硬件架构特性相匹配，才能最大化性能收益。无论是任务一中的 Cache 优化，还是任务二中的指令级并行优化，都展示了软件层面如何通过调整数据访问模式和指令调度来更好地利用 CPU 的缓存层次结构和超标量执行能力。

- 任务一中，在时间复杂度完全相同的条件下，程序的访存模式如果能够迎合物理内存的存储顺序，就能极大提升各级 Cache 的命中率。
- 任务二中，在连续密集的计算任务中，为了提高性能，必须减少数据之间的依赖性，通过超标量的策略调用 CPU 的多条执行链路来并行处理指令，从而提升整体吞吐量。